

实习报告

题目：2.4马踏棋盘，6.4平衡二叉树操作的演示

班级：2406

姓名：张晨轩，沈祺涵，王衍智

学号：2024K8009929011，2024K8009929003，2024K8009929005

完成日期：2026.06.11

2.4 马踏棋盘

1. 需求分析

- 输入的形式和输入值的范围：**
读者可自行指定一个马的初始位置 (i, j) ，坐标范围为 $0 \leq i, j \leq 7$ 。
- 输出的形式：**
求出马的行走路线后，将数字 1, 2, ..., 64 依次填入一个 8×8 的方阵并输出；在可视化模式下，支持演示寻找行走路线的回溯过程。
- 程序所能达到的功能：**
将马随机放在 8×8 棋盘的某个方格中，按走棋规则移动，每个方格只进入一次，走遍全部 64 个方格。程序需采用**非递归**实现。此外支持选作功能：求出多条以至全部行走路线；探讨每次选择位置的“最佳策略”以减少回溯次数。
- 测试数据：**
由读者指定，可自行指定任意合法的 (i, j) 进行测试。

2. 概要设计

本程序采用非递归的深度优先搜索 (DFS) 结合 Warnsdorff 启发式策略。下面列出程序中使用的抽象数据类型定义、主程序流程以及各程序模块间的层次与调用关系：

2.1 抽象数据类型定义

程序中主要定义了用于模拟系统栈的栈节点结构体 `StackNode`，其结构如下：

```
1 typedef struct {
2     int x, y;    // 马的当前坐标
3     int dir;     // 马下一步的方向索引（在排序后的候选列表中的位置）
4 } StackNode;
```

基于此结构体，程序手动维护了一个容量为 64 的显式栈用于控制 DFS 的前进与回溯状态：

```
1 static StackNode stack[64]; // 棋盘最多走64步
2 static int top = -1;
```

2.2 主程序的流程

主程序 `main` 的整体控制流如下：

1. **环境与界面初始化**：设置控制台编码，打印程序的欢迎界面与基本说明。
2. **接收用户输入**：循环读取并校验用户输入的初始坐标 `(i, j)`，确保其在 $0 \sim 7$ 的合法范围内。
3. **运行参数配置**：提示用户选择 4 种运行模式之一（单条路径、多条路径、过程可视化等），并根据用户的输入设定目标求解数量 `maxSolutions` 和是否开启可视化的标识 `verbose`。
4. **执行核心搜索**：记录开始时间，调用核心搜索模块 `solve(startX, startY)` 进行路径求解。
5. **输出结果与统计**：搜索执行完毕后，检查是否为用户手动中断，并向控制台打印最终的汇总统计信息（包括找到的路径总数、探索节点总数、回溯总次数以及算法耗时）。

2.3 各程序模块之间的层次（调用）关系

程序采用模块化设计，以 `main` 函数为入口，核心逻辑集中在 `solve` 函数中。整体调用层次关系如下：

- `main()`：主函数（顶层），负责交互逻辑、环境配置、耗时统计以及结果展示。
 - `clearBuffer()`：在处理非法输入时被 `main` 调用，用于清理输入缓冲区防止死循环。
 - `solve(int startX, int startY)`：被 `main` 调用，是程序的**核心模块**。负责执行带有启发式剪枝的非递归 DFS。
 - `push() / pop() / isEmpty()`：被 `solve` 频繁调用，用于维护显式栈 `stack`，模拟函数调用栈的入栈、出栈与判空操作。
 - `centerDistance(int x, int y)`：被 `solve` 调用，用于计算候选格子到棋盘中心的曼哈顿距离，为候选格子的启发式排序提供次键比较依据。
 - `checkAbort()`：被 `solve` 周期性调用，通过检测系统底层键盘事件，判断用户是否按下了 `Q` 键从而实现安全中断。
 - `printBoard()`：被 `solve` 调用，当栈深度达到 64（即找到一条完整路线）时，输出 8×8 的数字结果方阵。
 - `printPathBoard()`：在开启可视化模式（`verbose` 为真）时被 `solve` 调用，用于逐步绘制带有当前状态的 ASCII 字符画棋盘。
 - `clearBuffer()`：在可视化模式下暂停并等待用户输入回车时，被 `solve` 辅助调用。

3. 详细设计

3.1 候选选择策略（Warnsdorff 启发式）

为减少无效搜索，程序在每一步展开前，会计算所有合法下一步候选格子的“出度”（即下一步还能走的其他合法格子数量），并将候选格子按照启发式规则进行排序。排序的主键是“出路的度数越小越好”，次键是“距离棋盘中心越远越好”，这极大降低了陷入死局的概率。核心代码实现如下：

```
1 // 收集所有合法的下一步候选格子
2 for (int i = 0; i < 8; i++) {
3     int nx = cur->x + dx[i];
4     int ny = cur->y + dy[i];
5     if (nx >= 0 && nx < 8 && ny >= 0 && ny < 8 && board[nx][ny] == 0) {
6         int deg = countDegree(nx, ny);
7         candidates[count].x = nx;
```

```

8     candidates[count].y = ny;
9     candidates[count].deg = deg;
10    count++;
11 }
12 }
13
14 // Warnsdorff 启发式排序:
15 // 优先选择度数小（出路少）的格子；度数相同时，优先选择距离中心较远的格子
16 for (int i = 0; i < count - 1; i++) {
17     for (int j = i + 1; j < count; j++) {
18         int d1 = centerDistance(candidates[i].x, candidates[i].y);
19         int d2 = centerDistance(candidates[j].x, candidates[j].y);
20         if (candidates[j].deg < candidates[i].deg ||
21             (candidates[j].deg == candidates[i].deg && d2 > d1)) {
22             // 交换位置
23             Candidate temp = candidates[i];
24             candidates[i] = candidates[j];
25             candidates[j] = temp;
26         }
27     }
28 }

```

3.2 死路预判（剪枝）

在遍历排序好的候选格子前，程序会进行死路预判。如果某个候选格子的出度为 0，并且当前搜索尚未达到 63 步（即并非跳向最后一步终点），说明走入该格后将陷入死胡同，此时程序直接将其剪枝跳过：

```

1 // 遍历排序后的候选列表
2 for (int i = cur->dir + 1; i < count; i++) {
3     int nx = candidates[i].x;
4     int ny = candidates[i].y;
5     int deg = candidates[i].deg;
6
7     // 死路剪枝：如果无路可走且还没到倒数第一步，必然是死局，直接跳过
8     if (deg == 0 && step < 63) continue;
9
10    // ... 执行进栈和状态转移逻辑
11 }

```

3.3 回溯机制的核心控制

程序通过手动维护栈 `stack` 来代替系统的函数调用栈。栈中每个节点不仅保存了马的坐标 `(x, y)`，还记录了已经尝试过的方向索引 `dir`。若在某一层中所有的候选方向都尝试失败（或被剪枝），程序将执行恢复现场、步数递减和弹栈的操作，回退到上一状态继续探索：

```

1 int moved = 0;
2 // 接着上次尝试的方向继续向后遍历
3 for (int i = cur->dir + 1; i < count; i++) {
4     // ... [进行前述的剪枝操作] ...
5
6     cur->dir = i; // 记录当前栈节点已尝试到第 i 个候选
7     push((StackNode){nx, ny, -1}); // 将新的位置作为新节点入栈

```

```

8     board[nx][ny] = ++step;           // 更新棋盘记录并增加步数
9     moved = 1;                        // 标记已成功找到下一步
10    break; // 深入下一层，中止当前节点的水平扩展
11 }
12
13 // 回溯处理：若当前节点无法向任何方向移动
14 if (!moved) {
15     board[cur->x][cur->y] = 0; // 撤销当前位置的棋盘标记
16     pop();                    // 弹出当前节点
17     step--;                  // 步数回退
18 }

```

3.4 多路线与可视化支持

当步数 `step` 累加至 64 时，说明已成功找到一条遍历整个棋盘的解。为了支持多路径模式（或穷举全部解），程序在输出当前解后，会主动模拟一次“回溯”（即将栈顶节点弹出，撤销最后一步），以迫使算法继续寻找其他可能的分支。同时程序加入了键盘侦听，以便在海量搜索时能够被用户随时安全中断：

```

1 // 检查是否走满 64 步，找到一条完整路径
2 if (step == 64) {
3     solutionsFound++;
4     printBoard(); // 输出当前解
5
6     // 如果运行模式设定为只寻找一条路径，直接返回成功状态
7     if (maxSolutions == 1) return 1;
8
9     // 检测用户是否按下了 'Q' 键中止程序
10    if (checkAbort()) { userAbort = 1; break; }
11
12    // 寻找多条路径的核心策略：主动回退一步，继续深搜下一个分支
13    board[stack[top].x][stack[top].y] = 0;
14    pop();
15    step--;
16    continue;
17 }

```

4. 调试分析

4.1 算法复杂度分析

- **时间复杂度**：国际象棋 8×8 棋盘的骑士周游路线总数超过 260 亿条。如果不加任何优化地进行暴力深度优先搜索，最坏情况下的时间复杂度高达 $O(8^{64})$ ，枚举整个状态空间是不现实的。但是，在本程序中引入 Warnsdorff 启发式策略后，对 8×8 棋盘极为有效。实测表明，几乎所有的起点都能在零回溯或极少量回溯的情况下瞬间找到第一条完整路径。
- **空间复杂度**：由于程序按要求采用了非递归的实现方式，避免了系统调用栈的深度递归带来的栈溢出风险。程序仅显式地分配了一个容量为 64 的 `StackNode` 数组作为辅助栈，因此空间复杂度为严格的 $O(64)$ （常数级）。

```

1 // 空间复杂度 O(64) 的代码体现
2 static StackNode stack[64]; // 棋盘最多走64步
3 static int top = -1;

```

4.2 启发式策略（Warnsdorff规则）的效能分析

在调试初期若仅使用传统的按固定顺序（如顺时针探索 8 个方向）的回溯法，程序很容易在最后几步陷入死局，产生海量的回溯。通过引入排序机制，算法性能得到了质的飞跃：

- 主键优化（出路的度数）**：优先走“出路最少”的格子。这种贪心策略直观上保证了棋盘边缘和角落的格子能够被尽早访问，防止后期这些位置成为无法到达的死胡同。
- 次键优化（中心距离度量）**：当多个候选格子的出路度数相同时，程序通过计算放大的曼哈顿距离 $\text{abs}(2 * x - 7) + \text{abs}(2 * y - 7)$ ，优先让马走向距离棋盘中心更远的位置。这一策略进一步降低了算法在棋盘中央陷入死胡同的概率。

4.3 剪枝与状态监控分析

在寻找多条路径（或不限数量模式）的调试过程中，由于解的空间极为庞大，程序很容易陷入长时间的搜索。对此，本程序做了以下调试和控制设计：

- 死路预判剪枝**：如果一个未走完 63 步的候选位置出度为 0（即 `deg == 0 && step < 63`），程序会将其判定为死路并直接剪枝，跳过入栈操作。这有效削减了无效的搜索分支。
- 安全中断机制**：在死循环或海量搜索时，程序利用 `GetAsyncKeyState('Q')` 实现了非阻塞的键盘侦听。在调试和实际运行中，用户按下 Q 键即可优雅退出并输出当前的耗时、节点和回溯统计信息，避免了强制结束进程导致的数据丢失。

```

1 // 检查用户是否按 Q 中止
2 if (checkAbort()) { userAbort = 1; break; }

```

- 耗时精确统计**：程序在搜索入口处利用 `<time.h>` 库的 `clock()` 函数记录 `t0` 和 `t1`，从而精确输出搜索耗费的 CPU 时间，为评估算法效能提供了直观的数据支撑。

5. 用户使用说明

- 编译程序**：程序为纯 C (C99/C11)，使用 GCC 编译：

```
gcc horsechess.c -o horsechess.exe -Wall -wextra
```

- 运行程序**：执行 `./horsechess.exe`。
- 程序提供 4 种运行模式：

模式	说明
[1]	寻找一条路径 — 使用 Warnsdorff 启发式快速求解
[2]	寻找多条路径 — 指定数量，输入 0 表示不限（搜完或按 Q 中止）
[3]	回溯过程可视化 — 逐步演示每一步的决策和回溯
[4]	可视化 + 多条路径 — 结合 2 和 3

提示：任意模式下均可按 Q 键随时中止搜索，程序会输出已找到的结果和统计数据。

6. 测试结果

- 测试用例 1（寻找一条路径）：

```
=====
          马踏棋盘 (Knight's Tour)
        Warnsdorff 启发式 + 回溯算法
=====

在 8x8 的国际象棋棋盘上，马按"日"字走法，
每个方格只进入一次，走遍全部 64 格。

请输入马的初始坐标 (i j, 范围 0~7, 空格分隔): 0 0

请选择运行模式：
[1] 寻找一条路径
[2] 寻找多条路径 (指定数量，输入 0 = 不限，搜完为止)
[3] 回溯过程可视化 (逐步演示寻找一条路径)
[4] 回溯可视化 + 寻找多条路径
>> 1
已找到：1 条 | 探索节点：63 | 回溯：0 | 耗时：0.0 秒
===== 解法 #1 =====
   1   4  35  20  61   6  43  22
  34  19   2   5  42  21  62   7
   3  36  49  64  51  60  23  44
  18  33  54  41  48  63   8  59
  37  14  47  50  55  52  45  24
  32  17  40  53  46  27  58   9
  13  38  15  30  11  56  25  28
  16  31  12  39  26  29  10  57

===== 搜索完毕 =====
起始位置： (0, 0)
找到路径数：1
探索节点数：63
回溯次数： 0
耗时：      0.010 秒
```

- 测试用例 2（寻找多条路径+回溯可视化）：

```
=====
      马踏棋盘 (Knight's Tour)
      Warnsdorff 启发式 + 回溯算法
=====

在 8x8 的国际象棋棋盘上，马按"日"字走法，
每个方格只进入一次，走遍全部 64 格。

请输入马的初始坐标 (i j, 范围 0~7, 空格分隔): 3 4

请选择运行模式:
[1] 寻找一条路径
[2] 寻找多条路径 (指定数量, 输入 0 = 不限, 搜完为止)
[3] 回溯过程可视化 (逐步演示寻找一条路径)
[4] 回溯可视化 + 寻找多条路径
>> 4
请输入想要寻找的路径数量 (0 = 不限, 直到搜完或按 Q 中止): 2

[前进] (3,4) --> (1,5), 候选中排第 1/8 (deg=5, dist=8)

+-----+
| . . . . . |
| . . . . K . |
| . . . . . |
| . . . * . . |
| . . . . . |
| . . . . . |
| . . . . . |
| . . . . . |
+-----+
步数: 2/64, 栈深度: 2
按回车继续 (输入 q 退出可视化)...

[前进] (1,5) --> (0,7), 候选中排第 1/5 (deg=1, dist=14)

+-----+
| . . . . . K |
| . . . . * . |
| . . . . . |
| . . . * . . |
| . . . . . |
| . . . . . |
| . . . . . |
| . . . . . |
+-----+
步数: 3/64, 栈深度: 3
按回车继续 (输入 q 退出可视化)...
```


[前进] (0,7) --> (2,6), 候选中排第 1/1 (deg=4, dist=8)

```
+-----+
| . . . . . * |
| . . . . . * . |
| . . . . . K . |
| . . . . . * . |
| . . . . . . . |
| . . . . . . . |
| . . . . . . . |
| . . . . . . . |
+-----+
```

步数: 4/64, 栈深度: 4
按回车继续 (输入 q 退出可视化)...

[前进] (2,6) --> (0,5), 候选中排第 1/4 (deg=3, dist=10)

```
+-----+
| . . . . . K . * |
| . . . . . * . . |
| . . . . . * . . |
| . . . . . * . . |
| . . . . . . . . |
| . . . . . . . . |
| . . . . . . . . |
| . . . . . . . . |
+-----+
```

步数: 5/64, 栈深度: 5
按回车继续 (输入 q 退出可视化)...

[前进] (0,5) --> (1,7), 候选中排第 1/3 (deg=2, dist=12)

```
+-----+
| . . . . . * . * |
| . . . . . * . K |
| . . . . . * . . |
| . . . . . * . . |
| . . . . . . . . |
| . . . . . . . . |
| . . . . . . . . |
| . . . . . . . . |
+-----+
```

步数: 6/64, 栈深度: 6
按回车继续 (输入 q 退出可视化)... q

已找到: 1 条 | 探索节点: 63 | 回溯: 0 | 耗时: 12.9 秒

===== 解法 #1 =====

18	15	34	53	20	5	36	3
33	54	19	16	35	2	21	6
14	17	60	55	52	49	4	37
59	32	51	48	1	56	7	22
28	13	58	61	50	47	38	43
31	64	29	46	57	42	23	8
12	27	62	41	10	25	44	39
63	30	11	26	45	40	9	24

已找到: 2 条 | 探索节点: 66 | 回溯: 2 | 耗时: 13.0 秒

===== 解法 #2 =====

18	15	34	53	20	5	36	3
33	54	19	16	35	2	21	6
14	17	60	55	52	49	4	37
59	32	51	48	1	56	7	22
28	13	58	61	50	47	38	43
31	62	29	46	57	42	23	8
12	27	64	41	10	25	44	39
63	30	11	26	45	40	9	24

===== 搜索完毕 =====

起始位置: (3, 4)

找到路径数: 2

探索节点数: 66

回溯次数: 2

耗时: 12.973 秒

- **测试用例 3（不限模式中断）：**输入起点 0 0，指定数量 0。程序持续搜索全部路线，按下 Q 键后搜索中断

```
===== 解法 #12979 =====
60 11 30 27 50 13 32 17
29 26 61 12 31 16 63 14
10 59 28 49 62 51 18 33
25 40 55 52 43 48 15 64
58 9 44 41 56 53 34 19
39 24 57 54 47 42 1 4
8 45 22 37 6 3 20 35
23 38 7 46 21 36 5 2

已找到：12980 条 | 探索节点：518688 | 回溯：505646 | 耗时：184.3 秒
===== 解法 #12980 =====
54 11 30 27 56 13 32 17
29 26 55 12 31 16 57 14
10 53 28 49 64 59 18 33
25 40 63 60 43 48 15 58
52 9 44 41 50 61 34 19
39 24 51 62 47 42 1 4
8 45 22 37 6 3 20 35
23 38 7 46 21 36 5 2

===== 解法 #12981 =====
54 11 30 27 56 13 32 17
29 26 55 12 31 16 57 14
10 53 28 49 62 59 18 33
25 40 63 60 43 48 15 58
52 9 44 41 50 61 34 19
39 24 51 64 47 42 1 4
8 45 22 37 6 3 20 35
23 38 7 46 21 36 5 2

[用户中止] 已按 Q 键，搜索中断。

===== 搜索完毕 =====
[用户中止] 搜索被手动中断。
起始位置： (5, 6)
找到路径数： 12981
探索节点数： 518691
回溯次数： 505648
耗时： 184.458 秒
```

- **全面测试：**对全部 $8 \times 8 = 64$ 个可能的起始位置逐一测试，**64/64 PASS**，全部成功找到合法骑士周游路径，并经脚本验证合法。

7. 附录

- 源文件： `/src/HorseChess/horsechess.c`

6.4 平衡二叉树操作的演示

1. 需求分析

- **输入的形式和输入值的范围：**
操作界面给出查找、插入和删除三种操作供选择。对于选作的分裂操作，需输入一个整型参考值 x 。
- **输出的形式：**
每次插入或删除一个结点后，更新平衡二叉树的显示。树的显示采用凹入表形式，在终端中打印出带有缩进层级和平衡因子的树状结构。
- **程序所能达到的功能：**

利用平衡二叉树实现一个动态查找表，支持查找、插入和删除三大基础功能。针对选作内容，程序能够合并两棵平衡二叉树，以及把一棵平衡二叉树分裂为两棵（一棵所有关键字 $\leq x$ ，另一棵所有关键字 $> x$ ）。程序同时提供了基于严格平衡的 AVL 树和期望平衡的无旋 Treap (FHQ Treap) 两种不同方式的实现来完成选做任务。

- **测试数据：**

由读者自行设定输入测试的节点关键字序列和操作指令。

2. 概要设计

2.1 抽象数据类型定义

为了避免繁琐的指针操作和动态内存碎片，程序采用数组（`std::vector`）加静态内存池的方式来存储树节点。定义了两种树节点结构：

```
1 // 1. AVL 树节点结构（严格平衡）
2 struct TreeNode{
3     int key;    // 键值
4     int height; // 绝对高度
5     int bf;     // 平衡因子 = 右子树高度 - 左子树高度
6     int left;   // 左孩子下标
7     int right;  // 右孩子下标
8 };
9
10 // 2. FHQ Treap 节点结构（期望平衡，用于选作）
11 struct TreapNode {
12     int key;      // 键值
13     int priority; // 随机优先级（维护堆性质）
14     int size;     // 子树大小
15     int left;
16     int right;
17 };
```

为实现节点的循环利用，程序维护了一个删除节点回收站。每次新建节点优先从回收站取用废弃下标，避免无限扩容。

2.2 主程序的流程

主程序 `main` 以菜单驱动的方式运行，整体控制流如下：

1. **初始化：**配置控制台 UTF-8 编码，初始化全局内存池，加入 0 号节点作为空树的哨兵。
2. **状态展示：**在每个交互循环的开头，调用 `showAVLTree()` 打印当前 AVL 树的形态。
3. **接受指令：**提供包括插入、删除、查找、基于 Treap 的分裂合并演示、基于 AVL 的分裂合并演示以及退出等操作菜单。
4. **功能分发：**根据用户选择，读取相应参数并调用 `tree/AVL.cpp` 或 `tree/treap.cpp` 中的核心算法，反馈操作结果。

3. 详细设计

3.1 AVL树的插入与旋转调整

在执行普通的二叉搜索树插入后，程序向上回溯更新节点的高度（`height`）和平衡因子（`bf = 右树高 - 左树高`）。若判定 `bf < -1`，说明左子树过高，再根据左孩子的平衡因子进一步判断是单次右旋还是先左旋后右旋（LR 旋转）；右侧过高同理。

```
1      updateNode(idx); // 更新插入后的节点信息
2
3      if(tree[idx].bf < -1){ // 左侧更高
4          if(tree[tree[idx].left].bf <= 0){
5              return rightRotate(idx); // 单次右旋处理 LL 型不平衡
6          }else{
7              int newLeft = leftRotate(tree[idx].left); // LR 型不平衡
8              tree[idx].left = newLeft;
9              return rightRotate(idx);
10         }
11     }
12     // ... 对称处理右侧更高的 LL 和 RL 旋转 ...
```

3.2 AVL树的删除与替代机制

删除算法采用了类似 B 树的替代删除策略。如果待删节点只有一棵子树或为叶子节点，则直接将其删除并由子节点顶替。如果待删节点有两棵子树，程序会进入右子树一直向左找，找到它的**中序直接后继**。将后继的键值覆盖到当前节点，随后递归地在右子树中删除该后继节点，最后向上回溯更新并旋转：

```
1         }else{ // 如果两棵子树
2             // 找它的中序直接后继（右子树的最左节点）
3             int successor = tree[idx].right;
4             while(tree[successor].left){
5                 successor = tree[successor].left;
6             }
7             tree[idx].key = tree[successor].key; // 将后继复制过来
8
9             // 递归删除右子树的后继节点
10            int nextRight = removeNode(tree[idx].right, tree[successor].key);
11            tree[idx].right = nextRight;
12        }
```

3.3 选作：树的分裂与合并的两种实现方式

本程序对选作内容“树的分裂与合并”给出了两套截然不同的实现方案：

方案一：基于 AVL 树的严格平衡重构

设计思路：遍历源树，将节点逐个拆下，并使用 `insertNode` 插入到目标树中（分裂时根据值域判断插入哪棵新树）。

```
1 // AVL 分裂：前序遍历原树，将节点根据 val 判断插入到左树或右树中
2 static void splitTraversal(int srcNode, int val, int& outLeft, int& outRight){
3     if(!srcNode) return;
```

```

4     int leftChild = tree[srcNode].left;
5     int rightChild = tree[srcNode].right;
6     bool dummy = false;
7
8     if(tree[srcNode].key <= val)
9         outLeft = insertNode(outLeft, tree[srcNode].key, dummy);
10    else
11        outRight = insertNode(outRight, tree[srcNode].key, dummy);
12
13    deleteNode(srcNode);
14    splitTraversal(leftChild, val, outLeft, outRight);
15    splitTraversal(rightChild, val, outLeft, outRight);
16 }
17
18 // AVL 合并: 遍历将一棵树的节点逐一插入到另一棵树中
19 static int mergeTraversal(int destRoot, int srcNode){
20     if(!srcNode) return destRoot;
21     int leftChild = tree[srcNode].left;
22     int rightChild = tree[srcNode].right;
23     bool dummy = false;
24
25     destRoot = insertNode(destRoot, tree[srcNode].key, dummy);
26     deleteNode(srcNode);
27     destRoot = mergeTraversal(destRoot, leftChild);
28     destRoot = mergeTraversal(destRoot, rightChild);
29     return destRoot;
30 }

```

方案二：基于 FHQ Treap 的期望平衡拆接

设计思路：无旋 Treap 本身就是由 `Split` 和 `Merge` 两个基础操作构建的。分裂时，根据二叉搜索树的性质直接断开并重接指针；合并时，根据随机生成的 `priority` 维护大根堆性质重接两棵树，期间不涉及任何旋转操作。

```

1 // Treap 分裂: 直接根据权值打断并重连子树指针
2 pair<int, int> FHQ_split(int root, int val){
3     if(!root) return {0, 0};
4     if(fhq_tree[root].key <= val){
5         pair<int, int> p = FHQ_split(fhq_tree[root].right, val);
6         fhq_tree[root].right = p.first; // 将<=val的部分接在第一棵树上
7         updateTreapNode(root);
8         return {root, p.second};
9     }else{
10        pair<int, int> p = FHQ_split(fhq_tree[root].left, val);
11        fhq_tree[root].left = p.second;
12        updateTreapNode(root);
13        return {p.first, root};
14    }
15 }
16
17 // Treap 合并: 根据 priority 维护堆性质重连
18 int FHQ_merge(int x, int y){
19     if(!x || !y) return x ? x : y;
20     if(fhq_tree[x].priority > fhq_tree[y].priority){

```

```

21         fhq_tree[x].right = FHQ_merge(fhq_tree[x].right, y);
22         updateTreapNode(x);
23         return x;
24     }else{
25         fhq_tree[y].left = FHQ_merge(x, fhq_tree[y].left);
26         updateTreapNode(y);
27         return y;
28     }
29 }

```

4. 调试分析

4.1 复杂度分析

- **时间复杂度**：AVL 树的基础操作（查找、插入、删除）通过严密的平衡因子维护和旋转操作，保证了在最坏情况下严格的 $O(\log n)$ 性能。
- **空间复杂度**：采用 `std::vector` 实现的静态内存池，节点数为 n 时占用 $O(n)$ 空间，不会引发由于频繁 `new/delete` 导致的内存碎片。

4.2 AVL 与 Treap 分裂合并实现的优劣对比

本程序针对分裂和合并功能完整实现了 AVL 与 Treap 两种方案，通过对比调试得出以下分析：

- **AVL 树方案**：
 - **优势**：分裂或合并后，结果必定是一棵极其严格的平衡二叉树，保证了后续所有搜索、插入等操作的时间下限最优。代码复用率高，只需重复调用 `insertNode` 即可完成。
 - **劣势**：耗时巨大。将树 A 拆碎后逐一插入树 B 中，时间复杂度达到了 $O(m \log(n + m))$ ，在面对海量数据的大树分裂和合并时性能表现吃力。
- **FHQ Treap（无旋树堆）方案**：
 - **优势**：专为区间和集合的批量操作而生。由于只需要顺着路径重定向指针，不涉及任何单节点的重复开辟与插入，无论是 Split 还是 Merge 都直接拥有期望 $O(\log n)$ 的极高效率。
 - **劣势**：不再刻意维持左右子树高度差绝对 ≤ 1 的物理平衡，而是通过 `rand()` 优先级维护堆性质来达到“期望平衡”。在极其罕见的极端随机数下，树可能退化，得不到绝对性能保障。

5. 用户使用说明

1. **环境准备**：使用 CMake 和 MinGW 编译，仅支持 Windows 环境。确保终端中可执行 `gcc` 和 `cmake`。
2. **编译程序**：双击项目根目录下的 `cmake-build.bat` 批处理脚本即可一键编译。
3. **运行与交互**：运行编译出的 `build/BalancedTree.exe`。程序会显示控制台菜单，按照提示输入关键字即可完成相应操作并立即查看到凹入表渲染的结果。

6. 测试结果

- 先使用插入操作插入 0~9，建立一棵 AVL 树：

```
C:\Users\30130\Desktop\work x + v
----- 当前 AVL 树状态 -----
      9(bf:0)
     8(bf:1)
    7(bf:0)      6(bf:0)
      5(bf:0)
     4(bf:0)
3(bf:1)
  2(bf:0)
 1(bf:0)
 0(bf:0)

-----

请选择操作：
1. 插入结点 (Insert)
2. 删除结点 (Delete)
3. 查找结点 (Search)
4. [选作] 分裂、合并平衡二叉树 (基于 FHQ Treap)
5. [选作] 分裂、合并平衡二叉树 (基于 AVL)
0. 退出程序 (Exit)
>> |
```

• 删除节点6:

```
C:\Users\30130\Desktop\work x + v
----- 当前 AVL 树状态 -----
      9(bf:0)
     8(bf:1)
    7(bf:0)
     5(bf:-1)
      4(bf:0)
3(bf:1)
  2(bf:0)
 1(bf:0)
 0(bf:0)

-----

请选择操作：
1. 插入结点 (Insert)
2. 删除结点 (Delete)
3. 查找结点 (Search)
4. [选作] 分裂、合并平衡二叉树 (基于 FHQ Treap)
5. [选作] 分裂、合并平衡二叉树 (基于 AVL)
0. 退出程序 (Exit)
>> |
```

• 分别查找节点4和6:

```
C:\Users\30130\Desktop\work x + v
----- 当前 AVL 树状态 -----
      9(bf:0)
     8(bf:1)
    7(bf:0)
     5(bf:-1)
      4(bf:0)
3(bf:1)
     2(bf:0)
    1(bf:0)
     0(bf:0)
-----

请选择操作：
1. 插入结点 (Insert)
2. 删除结点 (Delete)
3. 查找结点 (Search)
4. [选作] 分裂、合并平衡二叉树 (基于 FHQ Treap)
5. [选作] 分裂、合并平衡二叉树 (基于 AVL)
0. 退出程序 (Exit)
>> 3
请输入要查找的关键词：4
[结果] 找到了关键词 4
Press any key to continue . . . |
```

```
C:\Users\30130\Desktop\work x + v
----- 当前 AVL 树状态 -----
      9(bf:0)
     8(bf:1)
    7(bf:0)
     5(bf:-1)
      4(bf:0)
3(bf:1)
     2(bf:0)
    1(bf:0)
     0(bf:0)
-----

请选择操作：
1. 插入结点 (Insert)
2. 删除结点 (Delete)
3. 查找结点 (Search)
4. [选作] 分裂、合并平衡二叉树 (基于 FHQ Treap)
5. [选作] 分裂、合并平衡二叉树 (基于 AVL)
0. 退出程序 (Exit)
>> 3
请输入要查找的关键词：6
[结果] 未找到了关键词 6
Press any key to continue . . . |
```

- 以4为分割点，分裂、合并两棵树：
 - FHQ_Treap合并后的树显然是不满足严格平衡的
 - AVL树在分裂、合并前后都满足平衡性质


```
C:\Users\30130\Desktop\work x + v
0(bf:0)

-----
请选择操作：
1. 插入结点 (Insert)
2. 删除结点 (Delete)
3. 查找结点 (Search)
4. [选作] 分裂、合并平衡二叉树 (基于 FHQ Treap)
5. [选作] 分裂、合并平衡二叉树 (基于 AVL)
0. 退出程序 (Exit)
>> 4
请输入分裂参考值 x (<=x 和 >x): 4

=== 分裂出的第一棵树 (<= 4) ===
      4(sz:2)
        3(sz:1)
2(sz:5)
  1(sz:2)
    0(sz:1)

=== 分裂出的第二棵树 (> 4) ===
      9(sz:1)
8(sz:4)
  7(sz:2)
    5(sz:1)

=== 两棵树合并后如下 ===
      9(sz:1)
8(sz:9)
  7(sz:4)
    5(sz:3)
      4(sz:2)
        3(sz:1)
2(sz:7)
  1(sz:2)
    0(sz:1)

(注：此操作在独立沙盒中演示，不影响原 AVL 树结构)
Press any key to continue . . . |
```

```
C:\Users\30130\Desktop\work x + v
7(bf:0)
  5(bf:-1)
    4(bf:0)
3(bf:1)
  2(bf:0)
    1(bf:0)
      0(bf:0)

-----
请选择操作：
1. 插入结点 (Insert)
2. 删除结点 (Delete)
3. 查找结点 (Search)
4. [选作] 分裂、合并平衡二叉树 (基于 FHQ Treap)
5. [选作] 分裂、合并平衡二叉树 (基于 AVL)
0. 退出程序 (Exit)
>> 5
请输入分裂参考值 x (<=x 和 >x): 4
=== 分裂出的第一棵树 (<= 4) ===
      4(bf:0)
    3(bf:0)
      2(bf:0)
1(bf:1)
  0(bf:0)

=== 分裂出的第二棵树 (> 4) ===
      9(bf:0)
    8(bf:1)
7(bf:1)
  5(bf:0)

=== 两棵树合并后如下 ===
      9(bf:0)
    8(bf:0)
      7(bf:0)
    5(bf:1)
      4(bf:0)
3(bf:1)
  2(bf:0)
    1(bf:0)
      0(bf:0)
Press any key to continue . . . |
```

7. 附录

- 所有头文件存在于 `src/BalancedTree/include/` 下，与代码文件一一对应
- 平衡树模块的主要源代码文件树结构（位于 `src/BalancedTree/src/`）如下：

```
1 | src/
2 |   |— common/
3 |   |   |— common.cpp      (静态内存池与节点空间管理)
4 |   |— display/
5 |   |   |— display.cpp     (AVL与Treap的凹入表格式化打印)
6 |   |— tree/
7 |   |   |— AVL.cpp         (AVL核心功能：插入、旋转、删除、前序遍历重构分裂合并)
8 |   |   |— treap.cpp       (FHQ Treap核心功能：基于堆性质的高效分裂与合并)
9 |   |— main.cpp           (交互式菜单与终端控制逻辑)
```